

Basic Java & DSA Practice**Placement preparation portfolio | VS Code-style Java syntax colouring**

Problems included: 52 | Your original code is shown without edits.

Colour key: keywords blue, types teal, strings orange, comments green, numbers light green.

Problem: Arrange Alphabets In Desc

Level: Basic | Source: Frequency problems/ArrangeAlphabetsInDesc.java

Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```
import java.util.Scanner;

public class ArrangeAlphabetsInDesc{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String res = "";

        for(int i=0;i<str.length();i++){

            char ch = str.charAt(i);
            if(res.indexOf(ch) == -1){
                res += ch;
            }
        }

        char[] arr = res.toCharArray();

        for(int i=0;i<arr.length-1;i++){
            for(int j=i+1;j<arr.length;j++){
                if(arr[i] < arr[j]){
                    char temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }

        for(char c : arr){
            System.out.print(c);
        }
    }
}
```

Problem: Counting Frequency

Level: Basic | Source: Frequency problems/CountingFrequency.java

Time: $O(n)$ | Space: $O(n)$

```
import java.util.Scanner;

public class CountingFrequency{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        int[] freq = new int[256];

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            freq[ch]++;
        }

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            // if(freq[ch] != 0){
            //     System.out.print(ch);
            // }
        }
    }
}
```

```

        // System.out.print(ch + ": " + freq[ch] + " ");
        // freq[ch] = 0;

        // }
    }
}

```

Problem: Rotate String N

Level: Basic | Source: Frequency problems/RotateStringN.java
 Time: $O(n)$ | Space: $O(1)$ auxiliary

```

import java.util.*;

public class RotateStringN {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        int n = sc.nextInt();

        int len = str.length();

        // Print from n to end
        for(int i = len-n; i < len; i++){
            System.out.print(str.charAt(i));
        }

        // Print from 0 to n-1
        for(int i = 0; i < str.length()-n; i++){
            System.out.print(str.charAt(i));
        }
    }
}

```

Problem: Square Matrix Corner Sum

Level: Basic | Source: Frequency problems/SquareMatrixCornerSum.java
 Time: $O(n^2)$ | Space: $O(n)$

```

import java.util.Scanner;

public class SquareMatrixCornerSum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[][] arr = new int[n][n];

        // Input
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        // Corner sum
        int sum = arr[0][0]
            + arr[0][n-1]
            + arr[n-1][0]
            + arr[n-1][n-1];

        System.out.print(sum);
    }
}

```

Problem: Downtriangle

Level: Basic | Source: pattern/Downtriangle.java
 Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```
import java.util.Scanner;
public class Downtriangle{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=n;i>=1;i--){
            for(int j=1;j<=i;j++){
                System.out.print(j);

            }
            System.out.println();
        }
    }
}
```

Problem: Half Diamond

Level: Basic | Source: pattern/HalfDiamond.java
 Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```
import java.util.Scanner;
public class HalfDiamond{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                System.out.print("* ");
            }
            System.out.println();
        }
        for(int i=n-1;i>=1;i--){
            for(int j=1;j<=i;j++){
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Problem: Largest

Level: Basic | Source: pattern/Largest.java
 Time: $O(n)$ | Space: $O(n)$

```
import java.util.Scanner;

public class Largest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int max = arr[0];

        for(int i=1;i<n;i++){
            if(arr[i]>max){
                max = arr[i];
            }
        }
        System.out.println(max);
    }
}
```

Problem: Reverse Triangle

Level: Basic | Source: pattern/ReverseTriangle.java
 Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```
import java.util.Scanner;

public class ReverseTriangle{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for(int i=0;i<n;i++)//row
        {
            for(int j=0;j<=i;j++){
                System.out.print(" "); //space
            }
            for(int k=i;k<n;k++){
                System.out.print("* "); // *
            }
            System.out.println();
        }
    }
}
```

Problem: Second Largest

Level: Basic | Source: pattern/SecondLargest.java

Time: O(n) | Space: O(n)

```
import java.util.Scanner;
public class SecondLargest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int largest = Integer.MIN_VALUE;
        int secondlargest = Integer.MIN_VALUE;
        for(int i=0;i<n;i++){
            if(arr[i]>largest){
                secondlargest = largest;
                largest = arr[i];
            }
        }

        for(int i=0;i<n-1;i++){
            if(arr[i]>secondlargest && secondlargest != largest){
                secondlargest = arr[i];
            }
        }
        System.out.println(secondlargest);
    }
}
```

Problem: Smallest

Level: Basic | Source: pattern/Smallest.java

Time: O(n) | Space: O(n)

```
import java.util.Scanner;

public class Smallest{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
```

```

        arr[i] = sc.nextInt();
    }

    int min = arr[0];
    for(int i=1;i<n;i++){
        if(arr[i]<min){
            min = arr[i];
        }
    }
    System.out.println(min);
}
}

```

Problem: Sumofarray

Level: Basic | Source: pattern/Sumofarray.java
 Time: O(n) | Space: O(n)

```

import java.util.*;
public class Sumofarray{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int sum = 0;
        for(int i=0;i<n;i++){
            sum += arr[i];
        }
        System.out.print(sum);
    }
}

```

Problem: binarypattern

Level: Basic | Source: pattern/binarypattern.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class binarypattern{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                if((i+j)%2==0){
                    System.out.print(1); //System.out.print(0);
                }
                else{
                    System.out.print(0); //System.out.print(1);
                }
            }
            System.out.println();
        }
    }
}

```

Problem: characterpattern

Level: Basic | Source: pattern/characterpattern.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;
public class characterpattern{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
    }
}

```

```

    char c = 'A';

    for(int i=1;i<=n;i++){
        for(int j=1;j<=i;j++){
            System.out.print(c);
            c++;
        }
        System.out.println();
    }
}

```

Problem: continuousnumber

Level: Basic | Source: pattern/continuousnumber.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;
public class continuousnumber{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int num = 1;
        int n = sc.nextInt();
        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                System.out.print(num);
                num++;
            }
            System.out.println();
        }
    }
}

```

Problem: fixednumpattern

Level: Basic | Source: pattern/fixednumpattern.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class fixednumpattern{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=1;i<=4;i++){
            for(int j=1;j<=4;j++){
                // System.out.print(j); //if we print j o/p is 1234 4 times
                System.out.print(i); //if we print i o/p is 1111 2222 3333 4444
            }
            System.out.println();
        }
    }
}

```

Problem: fulldiamond

Level: Basic | Source: pattern/fulldiamond.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class fulldiamond {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            for (int s = 1; s <= n - i; s++) {
                System.out.print(" ");
            }
        }
    }
}

```

```

    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print(j); // print "*"
    }
    System.out.println();
}

for (int i = n - 1; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) {
        System.out.print(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print(j); // print "*"
    }
    System.out.println();
}
}
}
}

```

Problem: hollow

Level: Basic | Source: pattern/hollow.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;
public class hollow{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                if(i==1 || i==n || j==1 || j==n)
                    System.out.print("*");
                else
                    System.out.print(" ");
            }
            System.out.println();
        }
    }
}

```

Problem: palindromepyramid

Level: Basic | Source: pattern/palindromepyramid.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class palindromepyramid {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int num = 1;

        for (int i = 1; i <= n; i++) {
            for (int s = 1; s <= n - i; s++) {
                System.out.print(" ");
            }
            for (int k = 1; k <= i; k++) {
                System.out.print(k);
            }
            for(int j=i-1;j>=1;j--)
                System.out.print(j);
            // num++;
            System.out.println();
        }
    }
}

```

```

    }
}

```

Problem: pattern

Level: Basic | Source: pattern/pattern.java
 Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```

import java.util.Scanner;
public class pattern{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                System.out.print(j);
            }
            System.out.println();
        }
    }
}

```

Problem: pyramid

Level: Basic | Source: pattern/pyramid.java
 Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```

import java.util.Scanner;
public class pyramid{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for(int i=1;i<=n;i++){
            for(int s=1;s<=n-i;s++){
                System.out.print(" ");
            }
            for(int j=1;j<=2*i-1;j++){
                System.out.print(j);
            }
            System.out.println();
        }
    }
}

```

Problem: reverse

Level: Basic | Source: pattern/reverse.java
 Time: $O(n^2)$ | Space: $O(1)$ auxiliary

```

import java.util.Scanner;
public class reverse{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for(int i=1;i<=4;i++){
            for(int j=i;j>=1;j--){
                System.out.print(j);
            }
            System.out.println();
        }
    }
}

```

Problem: samenumpattern


```
import java.util.Scanner;

public class samenumpattern{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=1;i<=4;i++){
            for(int j=1;j<=i;j++){
                System.out.print(i);
            }
            System.out.println();
        }
    }
}
```

Problem: space

Level: Basic | Source: pattern/space.java

Time: O(n?) | Space: O(1) auxiliary

```
import java.util.Scanner;
public class space{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n-i;j++){
                System.out.print(" ");
            }
            for(int k=1;k<=i;k++){
                System.out.print(k);
            }
            System.out.println();

            for(int i=n-1;i>=1;i--){
                for(int j=1;j<=n-i;j++){
                    System.out.print(" ");
                }
                for(int k=1;k<=i;k++){
                    System.out.print(k);
                }
                System.out.println();
            }
        }
    }
}
```

Problem: Advanced Rev

Level: Basic | Source: practice/AdvancedRev.java

Time: O(1) | Space: O(1) auxiliary

```
import java.util.Scanner;

public class AdvancedRev{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        StringBuilder sb = new StringBuilder(str);
        System.out.println(sb.reverse());
    }
}
```

Problem: Armstrong

Level: Basic | Source: practice/Armstrong.java

Time: O(n) | Space: O(1) auxiliary

```
import java.util.Scanner;
```

```

class Armstrong{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int original = n;
        int count = 0;
        int sum = 0;
        int digit = 0;
        int temp = n;

        while (temp!=0){
            count++;
            temp = temp/10;
        }
        temp = n;
        while(temp!=0){
            digit = temp%10;
            sum = sum+(int)Math.pow(digit,count);
            temp = temp/10;
        }

        if(sum == original){
            System.out.println("Armstrong");
        }
        else{
            System.out.println("not armstrong");
        }
    }
}

```

Problem: Array Basic

Level: Basic | Source: practice/ArrayBasic.java

Time: O(n) | Space: O(n)

```

import java.util.Scanner;

public class ArrayBasic{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        int sum = 0;

        // take first element first
        arr[0] = sc.nextInt();
        int max = arr[0];
        int min = arr[0];
        sum = arr[0];

        // loop from 1
        for(int i = 1; i < arr.length; i++){
            arr[i] = sc.nextInt();
            sum = sum + arr[i];

            if(arr[i] > max){
                max = arr[i];
            }

            if(arr[i] < min){
                min = arr[i];
            }
        }

        System.out.println("max: " + max);
        System.out.println("min: " + min);
    }
}

```

```

    System.out.println("sum: " + sum);

    // reverse print
    for(int i = arr.length - 1; i >= 0; i--){
        System.out.print(arr[i] + " ");
    }

    System.out.println();
    int counteven = 0;
    int countodd = 0;
    for(int i=0;i<arr.length;i++){
        if(arr[i]%2 == 0){
            counteven++;
        }
        else
            countodd++;
    }

    System.out.println("Even: " + counteven);
    System.out.println("Odd: " + countodd);

}
}

```

Problem: Automorphic

Level: Basic | Source: practice/Automorphic.java
 Time: O(1) | Space: O(1) auxiliary

```

import java.util.Scanner;

class Automorphic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int original = n;

        int square = n * n;

        int digits = 0;
        int temp = n;

        while (temp != 0) {
            digits++;
            temp = temp / 10;
        }

        int power = (int) Math.pow(10, digits);

        if (square % power == original) {
            System.out.println("Automorphic Number");
        } else {
            System.out.println("Not Automorphic Number");
        }
    }
}

```

Problem: Bubble Sort

Level: Basic | Source: practice/BubbleSort.java
 Time: O(n) | Space: O(n)

```

import java.util.Scanner;

public class BubbleSort{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
    }
}

```

```

int[] arr = new int[n];

for(int i=0;i<n;i++){
    arr[i] = sc.nextInt();
}

for(int i=0;i<n-1;i++){

boolean isswapped = true;

    //sorting the array
    for(int j=0;j<n-i-1;j++){
        if(arr[j] < arr[j+1]){ //descending order if ascending order arr[j] > arr[j+1]
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
            isswapped = true;
        }
    }

if(!isswapped){
    break;
}
}

//sorted array printing
for(int i=0;i<n;i++){
    System.out.print(arr[i] + " ");
}

System.out.println();

int Largest = arr[0];
int secLargest = -1;

//second largest element
for(int i=0;i<n;i++){
    if(arr[i] != Largest){
        secLargest = arr[i];
        break;
    }
}

if(secLargest == -1){
    System.out.println("NO Second Largest");
}else
    System.out.println("Second Largest: " + secLargest);

int smallest = arr[n-1];

int secSmallest = -1;

//second smallest
for(int i=n-2;i>=0;i--){
    if(arr[i] != smallest){
        secSmallest = arr[i];
        break;
    }
}

if(secSmallest == -1){
    System.out.println("NO Second smallest");
}else
    System.out.println("Second smallest: " + secSmallest);
}
}

```

Problem: Count Digits

Level: Basic | Source: practice/CountDigits.java

Time: O(n) | Space: O(1) auxiliary

```
import java.util.Scanner;

class CountDigits{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int count = 0;

        while(n!=0){
            n= n/10;
            count++;
        }
        System.out.println(count);
    }
}
```

Problem: Count Vowels

Level: Basic | Source: practice/CountVowels.java

Time: O(n) | Space: O(1) auxiliary

```
import java.util.Scanner;
public class CountVowels{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int count = 0;

        //instead of using str.charAt(i) again and again use
        //char ch = str.charAt(i);
        //if u want vowels checking easily convert string str.toLowerCase() then u will find vowels easily
        not tocheck for uppercase

        for(int i=0;i<str.length();i++){
            if(str.charAt(i)=='a' || str.charAt(i)=='e' || str.charAt(i)=='i' || str.charAt(i)=='o' || str.charAt(i)=='u' || str.charAt(i)=='A' || str.charAt(i)=='E' || str.charAt(i)=='I' || str.charAt(i)=='O' || str.charAt(i)=='U'){
                count++;
            }
        }
        System.out.print(count);
    }
}
```

Problem: Demo

Level: Basic | Source: practice/Demo.java

Time: O(n) average case | Space: O(n)

```
import java.util.*;

public class Demo{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] =sc.nextInt();
        }

        int target = sc.nextInt();
        boolean found = false;

        HashMap<Integer, Integer> map = new HashMap<>();

        for(int i=0;i<n;i++){
```

```

        int complement = target - arr[i];
        if(map.containsKey(complement)){
            found = true;
            System.out.println(map.get(complement) + " " + i);
            System.out.println(complement + " " + arr[i]);
            return;
        }

        map.put(arr[i],i);
    }
}

```

Problem: Factorial

Level: Basic | Source: practice/Factorial.java
 Time: O(n) | Space: O(1) auxiliary

```

import java.util.Scanner;

class Factorial{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int fact = 1;

        for(int i=1;i<=n;i++){
            fact = fact*i;
        }
        System.out.println(fact);
    }
}

```

Problem: Fibonacci

Level: Basic | Source: practice/Fibonacci.java
 Time: O(n) | Space: O(1) auxiliary

```

import java.util.Scanner;

class Fibonacci{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int a = 0;
        int b = 1;

        if(n==1){
            System.out.print(0);
        }else
            System.out.print(a + " " + b + " ");

        int c = 0;

        for(int i=3;i<=n;i++){
            c= a+b;
            a = b;
            b = c;
            System.out.print(c + " ");
        }
    }
}

```

Problem: First Non Repeat Char

Level: Basic | Source: practice/First_Non_Repeat_Char.java
 Time: O(n) average case | Space: O(n)

```
import java.util.*;

public class First_Non_Repeat_Char{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.next();

        HashMap<Character,Integer> map = new HashMap<>();

        for(char ch : str.toCharArray()){
            map.put(ch,map.getDefault(ch,0)+1);
        }

        for(char ch : str.toCharArray()){
            if(map.get(ch)==1){
                System.out.print(ch);
                break;
            }
        }
    }
}
```

Problem: Interview Level Rev

Level: Basic | Source: practice/InterviewLevelRev.java
Time: O(n) | Space: O(1) auxiliary

```
import java.util.Scanner;

public class InterviewLevelRev{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.next();
        String rev = "";

        for(int i=str.length()-1;i>=0;i--){
            rev= rev+str.charAt(i);
        }
        System.out.println(rev);
    }
}
```

Problem: Medium Level Reverse

Level: Basic | Source: practice/MediumLevelReverse.java
Time: O(n) | Space: O(1) auxiliary

```
import java.util.Scanner;

public class MediumLevelReverse{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();

        for(int i=str.length()-1;i>=0;i--){
            System.out.print(str.charAt(i));
        }
    }
}
```

Problem: Missing Number

Level: Basic | Source: practice/MissingNumber.java
Time: O(n) | Space: O(n)

```
import java.util.Scanner;
```

```

public class MissingNumber{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n-1];

        int sum = 0;

        for(int i=0;i<arr.length;i++){
            arr[i] = sc.nextInt();
            sum = sum+arr[i];
        }

        int total_sum = n * (n+1)/2;

        int missingNumber = total_sum - sum;

        System.out.print(missingNumber);
    }
}

```

Problem: Neon

Level: Basic | Source: practice/Neon.java

Time: O(n) | Space: O(1) auxiliary

```

import java.util.Scanner;

class Neon{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int original = n;

        int sqr = n*n;
        System.out.println("sqr " + sqr);
        int sum = 0;

        while(sqr!=0){
            int digit = sqr%10;
            sum = sum+digit;
            sqr = sqr/10;
        }

        if(sum == original)
            System.out.println("Neon number");
        else
            System.out.print("not neon number");
    }
}

```

Problem: Palindrome

Level: Basic | Source: practice/Palindrome.java

Time: O(n) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class Palindrome{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int original = n;
        int reverse = 0;

```



```

    while(n > 0){
        int last = n%10;
        n = n/10;
        reverse = reverse*10+last;
    }
    if(reverse == original)
        System.out.println("Palindrome");
    else
        System.out.println("not palindrome");
    }
}

```

Problem: Password

Level: Basic | Source: practice/Password.java

Time: O(n) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class Password{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        int n = str.length();

        boolean haslength = false;
        boolean hasUpper = false;
        boolean hasLower = false;
        boolean hasDigit = false;
        boolean hasSpecial = false;

        if(n>=9){
            haslength = true;
            for(int i=0;i<n;i++){
                char ch = str.charAt(i);

                if(Character.isUpperCase(ch)){
                    System.out.println("Perfect Password");
                    break;
                }else if(Character.isLowerCase(ch)){
                    System.out.println("Perfect Password");
                    break;
                }else if(Character.isDigit(ch)){
                    System.out.println("Perfect Password");
                }else{
                    hasSpecial = true;
                }
            }
        }
        if(haslength && hasUpper && hasLower && hasDigit && hasSpecial){
            System.out.println("Perfect Password");
        }else{
            System.out.println("Not a Perfect Password");
        }
    }else{
        System.out.println("Not a Perfect Password");
    }

    }sc.close();
}

```

Problem: Print Each Char

Level: Basic | Source: practice/PrintEachChar.java

Time: O(n) | Space: O(1) auxiliary

```

import java.util.Scanner;
public class PrintEachChar{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

```

```

String str = sc.nextLine();

for(int i=0;i<str.length();i++){
    System.out.println(str.charAt(i));
}
}
}

```

Problem: Reverse String

Level: Basic | Source: practice/ReverseString.java

Time: O(n) | Space: O(1) auxiliary

//it is baisc level reversing the string
 //first we convert the string to char array --> run loop on it from arr.length-1 to >=0 --> then print
 arr[i].

```
import java.util.Scanner;
```

```

public class ReverseString{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();

        char[] arr = str.toCharArray();

        for(int i=arr.length-1;i>=0;i--){
            System.out.print(arr[i]);
        }
    }
}

```

Problem: Sec Largest

Level: Basic | Source: practice/SecLargest.java

Time: O(n) | Space: O(n)

```
import java.util.Scanner;
```

```

public class SecLargest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int largest = arr[0];
        int Seclargest = 0;

        for(int i=0;i<n;i++){
            if(arr[i] > largest){
                Seclargest = largest;
                largest = arr[i];
            }

            if(arr[i] > Seclargest && arr[i] != largest){
                Seclargest = arr[i];
            }
        }

        System.out.println("largest: " + largest);
        System.out.println("seclargest: " + Seclargest);
    }
}

```

Problem: Star

Level: Basic | Source: practice/Star.java

```
import java.util.Scanner;

class Star{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int r = sc.nextInt();
        int c = sc.nextInt();

        for(int i=0;i<r;i++){
            for(int j=0;j<c;j++){
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Problem: Strong Number

Level: Basic | Source: practice/StrongNumber.java

Time: O(n) | Space: O(1) auxiliary

```
import java.util.Scanner;

class StrongNumber{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int original = n;

        int sum = 0;

        while(n!=0){
            int digit = n%10;
            int fact = 1;

            for(int i=1;i<=digit;i++){
                fact = fact*i;
            }
            sum = sum+fact;
            n = n/10;
        }

        System.out.println(sum);
        if(sum == original){
            System.out.println("Strong number");
        }
        else
            System.out.println("not Strong number");
    }
}
```

Problem: Arithmetic Expression

Level: Basic | Source: zoho/pattern/ArithmeticExpression.java

Time: O(n) | Space: O(1) auxiliary

```
import java.util.*;

public class ArithmeticExpression{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.next();

        Stack<Integer> stack = new Stack<>();
```

```

    for(int i=0;i<str.length();i++){
        char ch = str.charAt(i);

        if(Character.isDigit(ch)){
            stack.push(ch - '0');
        }else{
            int b = stack.pop();
            int a = stack.pop();

            int result = 0;

            if(ch == '+'){
                result = a+b;
            }else if(ch == '-'){
                result = a-b;
            }else if(ch == '*'){
                result = a*b;
            }else if(ch == '/') {
                result = a/b;
            }

            stack.push(result);
        }
    }
    // System.out.println(stack.pop());
    System.out.println(stack);
}
}

```

Problem: Inverted Left Traingle

Level: Basic | Source: zoho/pattern/Inverted_Left_Traingle.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class Inverted_Left_Traingle{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = 5;
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n-i+1;j++){
                System.out.print("*");
            }
            System.out.println("");
        }
    }
}

```

Problem: Left Triangle

Level: Basic | Source: zoho/pattern/Left_Triangle.java
 Time: O(n?) | Space: O(1) auxiliary

```

import java.util.Scanner;

public class Left_Triangle{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = 5;
        for(int i=0;i<n;i++){
            for(int j=0;j<=i;j++){
                System.out.print("*");
            }
            System.out.println("");
        }
    }
}

```

}

Problem: Number To Words

Level: Basic | Source: zoho/pattern/NumberToWords.java

Time: O(1) | Space: O(1) auxiliary

```
import java.util.Scanner;

public class NumberToWords{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String[] ones = {
            "", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine"
        };

        String[] teens = {
            "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Nineteen"
        };

        String[] tens = {
            "", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"
        };

        int num = sc.nextInt();

        if(num > 100){
            System.out.print(ones[num/100] + " Hundred ");
            num = num%100;
        }
        if(num >= 10 && num <= 19){
            System.out.print(teens[num - 10]);
        }else{
            System.out.print(tens[num/10] + " ");
            System.out.print(ones[num%10]);
        }

    }
}
```

Problem: Number Left Triangle

Level: Basic | Source: zoho/pattern/Number_Left_Triangle.java

Time: O(n?) | Space: O(1) auxiliary

```
import java.util.Scanner;

public class Number_Left_Triangle{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = 5;
        for(int i=1;i<=n;i++){
            for (int k = 1; k <= n - i; k++) {
                System.out.print(" ");
            }

            for(int j=1;j<=i;j++){
                System.out.print(j);
            }
            System.out.println("");
        }
    }
}
```

Problem: Solid Rect

Level: Basic | Source: zoho/pattern/Solid_Rect.java

Time: O(n?) | Space: O(1) auxiliary

```
import java.util.Scanner;

public class Solid_Rect{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Problem: Union Intersectn Except

Level: Basic | Source: zoho/pattern/Union_Intersectn_Except.java
 Time: O(n) average case | Space: O(n)

```
import java.util.*;

public class Union_Intersectn_Except{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[] a = new int[n];
        int[] b = new int[m];
        for(int i=0;i<n;i++){
            a[i] = sc.nextInt();
        }

        for(int i=0;i<m;i++){
            b[i] = sc.nextInt();
        }

        HashSet<Integer> set = new HashSet<>();

        for(int num : a){
            set.add(num);
        }

        for(int num : b){
            set.add(num);
        }

        System.out.println( "Union " + set);

        System.out.print("Intersection ");

        for(int num : b){
            if(set.contains(num)){
                System.out.print(num + " ");
            }
        }

        System.out.println();
        System.out.print("Except ");

        for(int num : a){
            if(num%2 != 0){
                System.out.print(num + " ");
            }
        }

        for(int num : b){
```

